Agentic Synth

Option 2: Text-to-Audio Synthesis

Direct waveform generation from natural language description

VST3/AU Plugin + Desktop App | Generative Audio

---

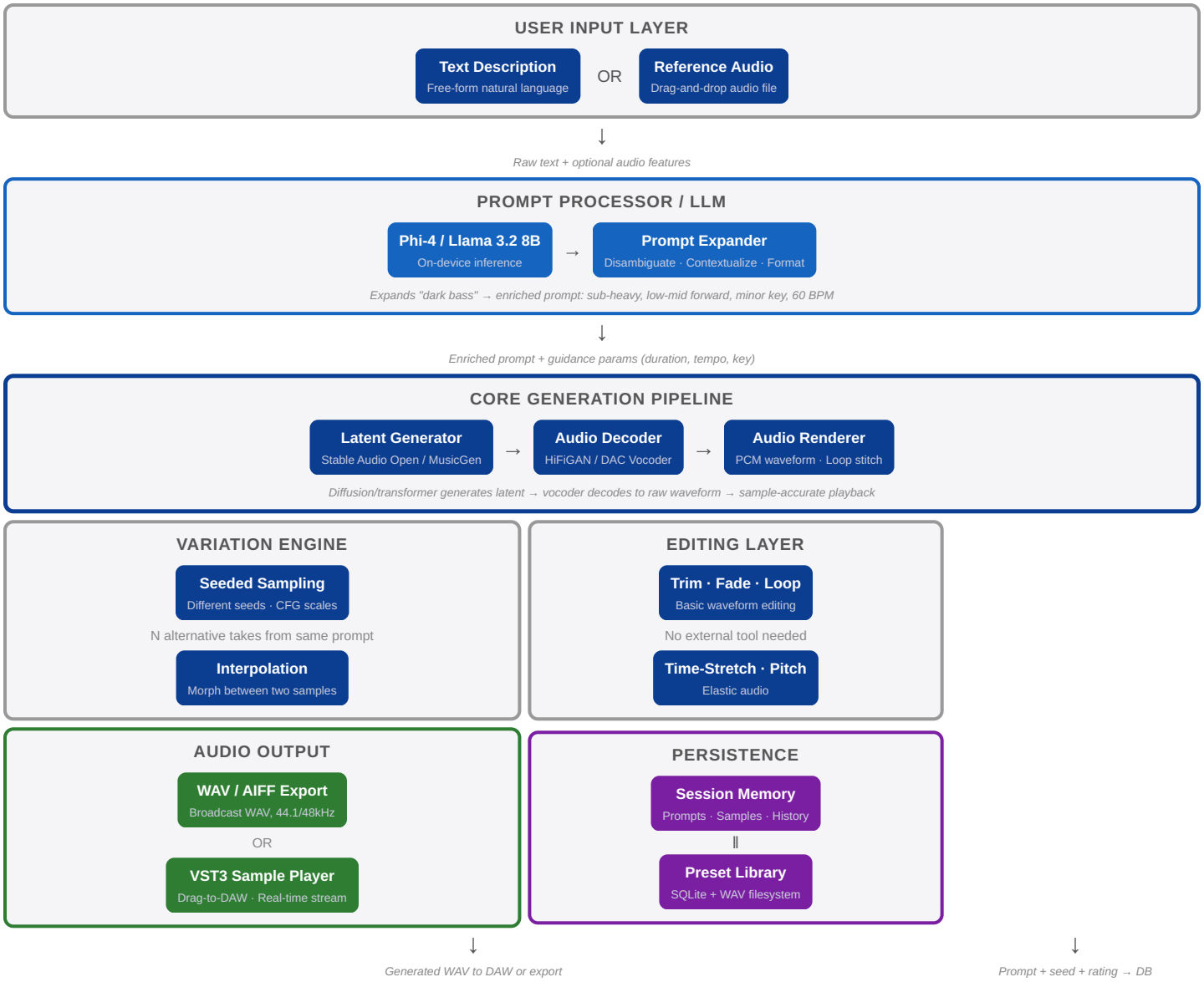
1. Overview

Option 1 (text-to-parameters) uses an LLM to translate descriptions into synth control parameters — oscillator tuning, filter cutoffs, envelopes, LFO routing — which drive a conventional synthesis engine. *Text → synth parameters → synth engine → audio.*

Option 2 (this document) eliminates the synth engine entirely. A generative audio model produces the **raw audio waveform directly** from the text description. No oscillators, no filters, no envelopes — just the sound itself, rendered by a model trained on real audio.

The producer describes what they hear in their imagination: *"a dark, ominous bass that feels large and otherworldly, with rhythmic vibrations that speed up and slow down, building to a crescendo like an earthquake."* The model generates the audio directly.

2. System Architecture



Key difference from Option 1: No synthesizer engine. No oscillators, filters, envelopes, or LFOs. The audio is generated directly by a diffusion/transformer model trained on real audio data. The trade-off: less precise control but can produce ANY sound — including evolving textures, realistic acoustics, and complex rhythms that a synth engine cannot create.

3. Component Descriptions

3.1 User Input Layer

- **Text Description** — free-form natural language. No templates or dropdowns. Short fragments ("dark bass, earthquake feel"), poetic descriptions, production directions ("build to a crescendo over 30 seconds") — all valid.
- **Reference Audio (optional)** — drag-and-drop audio file for the model to match in character/timbre/energy. If the generation model supports audio conditioning (Stable Audio does), features are extracted and used as additional guidance.

3.2 Prompt Processor / LLM

- **Model:** Phi-4 (3.8B) or Llama 3.2 8B — small, fast, runs on CPU alongside the GPU-accelerated audio model.
- Disambiguates vague terms ("dark" → "sub-heavy, low-mid forward, minimal highs, minor key"), adds production context (genre, BPM, key, mood), expands metaphors ("earthquake vibration" → "low-frequency tremolo, rumble 30-80Hz, rhythmic AM mimicking seismic waves").

- Formats the enriched prompt for the latent generator model, which expects audio-domain language, not metaphor.
- Incorporates session memory — if the user previously generated a related sound, carries that context forward.

3.3 Latent Generator Model

- The **core generative AI** — a diffusion or transformer model that has learned the mapping between text descriptions and audio waveforms.
- **Primary option: Stable Audio Open 2** — open-weight, generates up to 90s stereo audio at 44.1kHz from text prompts. Trained on Free Music Archive (CC-licensed). Runs on 6GB+ VRAM.
- **Alternative: MusicGen (Meta)** — good for musical sounds, single-channel, up to 30s. Smaller model sizes available (300M–3.3B params).
- Outputs a latent audio representation (compressed frequency-domain encoding) rather than raw PCM — the decoder converts this to waveform.

3.4 Audio Decoder / Vocoder

- Converts the latent representation into raw PCM audio waveform.
- **HiFiGAN** or **DAC (Descript Audio Codec)** decoder — fast, high quality, runs in near real-time on GPU.
- Outputs floating-point PCM samples ready for playback or export.

3.5 Audio Renderer

- Sample-accurate playback engine. Handles real-time streaming from the decoder to the audio output.
- Loop stitching — seamlessly loops generated audio by finding zero-crossing points.
- Multi-sample playback — layers separate generations (e.g., bass + pad + FX).

3.6 Variation Engine

- Generates N alternative takes from the same prompt by varying: random seed, classifier-free guidance scale, temperature in latent space, truncation threshold.
- Interpolation between two saved samples along a morph axis — creates smooth transitions.
- All variants rendered in background, shown as a grid for quick A/B comparison.

3.7 Editing Layer

- DAW-light editing: trim silence, fade in/out (linear/equal-power), set loop points, normalize gain.
- Basic EQ (low/high cut), time-stretch, pitch-shift (offline, high quality via rubberband or similar).
- The user should never need to open another tool just to trim silence or add a fade.

3.8 Audio Output

- **Export:** Broadcast WAV (44.1kHz/48kHz, 16/24-bit), AIFF, MP3 (320kbps).
- **DAW Integration:** VST3/AU sample player shell — loads generated WAV files into the DAW as a playable instrument. Drag-and-drop from the app directly into the DAW timeline.
- Real-time streaming via Rewire/Jack for live preview inside the DAW.

3.9 Session Memory & Preset Library

- SQLite database stores: prompt text, enriched prompt, model version, seed, CFG scale, generation time, user rating (liked/discarded), edit history.
- Audio files stored on filesystem at `~/.agentic-synth-samples/` with UUID filenames.
- Liked prompts boost similar generations in the future. Discarded prompts suppress them.
- Full-text search across all prompts and tags in the library.

---

4. Data Flow

"A dark, ominous bass with rhythmic earthquake vibrations building to a crescendo"

```
1. INPUT → User types or speaks the description
2. PROMPT PROCESSOR → Phi-4 expands it:
   "sub-heavy bass, 40-80Hz fundamental, minor key,
    rhythmic amplitude modulation (LFO rate accelerating
    from 0.5Hz to 8Hz over 24 seconds), distortion
    on attack, massive reverb tail, 30 seconds"
3. LATENT GENERATOR → Stable Audio Open runs diffusion:
   Text embedding → latent diffusion (30 steps) →
   latent audio representation (~12 seconds generation time)
4. DECODER → HiFiGAN converts latent → 44.1kHz PCM stereo
5. RENDER → Preview plays in app UI
6. USER SAYS → "Make it 15 seconds shorter"
7. REFINE → Prompt updated with new duration,
   model re-runs with changed guidance parameters
8. SAVE → User likes it → saved to library with
   prompt, seed, model version, tags
9. EXPORT → Drag WAV into Ableton Live,
   or open in VST3 sample player
```

---

5. Technology Choices

Component	Choice	Rationale
Core generation model	Stable Audio Open 2	Open-weight, 90s stereo at 44.1kHz, trained on CC-licensed data, runs on 6GB+ VRAM
Audio decoder	HiFiGAN / DAC	Fast, high-quality reconstruction from latent. Near real-time on GPU
Prompt LLM	Phi-4 or Llama 3.2 8B	Small, fast, runs on CPU alongside GPU audio model. No VRAM contention
Variation	Seed + CFG scale sampling	Simple, effective. Different seeds produce meaningfully different takes
DAW integration	VST3 sample player (JUCE)	Loads generated WAV as playable instrument in any DAW
Local hardware	RTX 3060+ / Apple M2+ (16GB RAM)	Generation target: <10s for 10s of audio. Quantized models reduce requirement
UI	React + TypeScript + WebSocket	Same as Option 1. Rich chat UX, fast iteration
Storage	SQLite + filesystem WAV	SQLite for metadata, WAV files on disk. No DB needed for large blobs

---

6. Key Risks & Mitigations

Generation Latency

Risk: 5-30 seconds per generation on consumer GPU breaks the creative flow. User can't iterate quickly.

Mitigation: Streaming generation (play first 2 seconds while rest continues), cached "quick start" prompt library, background pre-generation of 3 variants on idle, optional cloud GPU fallback for faster generation.

Quality Inconsistency

Risk: Model may not understand unusual descriptions ("glitchy granular bass that sounds like a dying robot"). Output quality varies with prompt specificity.

Mitigation: Prompt processor enriches vague descriptions. User rating feedback loops. Future: fine-tune model on synth/sound-design datasets for better understanding of sound design vocabulary.

Local Hardware Requirements

Risk: Needs GPU with 6GB+ VRAM. Many producers have laptops with integrated graphics only.

Mitigation: Quantized models (4-bit, 2GB VRAM), CPU fallback (slower but works), optional cloud inference tier with faster generation, progressive enhancement — basic features on lower hardware, full quality on GPU.

Lack of Precise Control

Risk: Unlike a synth where you can set "filter cutoff = 800Hz", text-to-audio is probabilistic. You can't say "reduce resonance by 15%."

Mitigation: Hybrid approach — Option 2 for initial sound, Option 1 for refinement. Audio-to-parameters analysis (spectral analysis of generated audio → parameter suggestions). Editing layer compensates (trim, EQ, fade, time-stretch).

Training Data / Copyright

Risk: Model may reproduce elements from training data (timbres, samples, recognizable audio).

Mitigation: Use models trained on licensed data (Stable Audio Open uses Free Music Archive — CC-by licensed). User indemnification, no commercial use of model outputs without verification, optional audio fingerprinting for copyright detection.

---

7. Implementation Phases

Phase 1 — Core Loop (Standalone App)

Goal: Text → audio in under 30 seconds. Proof of concept.

- Package Stable Audio Open behind a Python/FastAPI server
- Basic React UI: text input → generate → play → export WAV
- No prompt processor yet — raw text goes directly to model
- No editing, no variations, no session memory
- Target: 15s generation for 10s of audio on RTX 3060

Phase 2 — Prompt LLM + Variations

- Integrate Phi-4 for prompt expansion and enrichment
- Variation engine: seed sampling, CFG scale sweep
- SQLite session memory: prompt history, liked/discarded tracking
- Preset library browser with full-text search
- Basic editing: trim + fade

Phase 3 — DAW Integration

- VST3/AU sample player shell (JUCE) — loads generated WAV files
- Drag-and-drop from app to DAW timeline
- Plugin recall: DAW saves reference to generated sample
- Basic EQ and gain editing

Phase 4 — Advanced Features

- **Style transfer:** "Make this sound like [reference sample]" — audio-to-audio generation
- **Multi-track generation:** Generate bass + pad + FX simultaneously
- **Cloud GPU offload:** Faster generation via cloud, optional subscription tier
- **Real-time streaming:** Generate and play simultaneously, infinite evolving textures
- **Hybrid mode:** Option 2 generation → Option 1 synth analysis → tweak with parameters

---

8. Option 1 vs Option 2 — When to Use Which

Scenario	Option 1 (Parameters)	Option 2 (Direct Audio)
Classic synth sound (lead, bass, pad)	✅ Excellent — precise control	🟡 Good — may not match exact vision
Evolving, complex textures	❌ Hard — multi-LFO routing tedious	✅ Excellent — model handles evolution
Realistic / acoustic sounds	❌ Impossible — synth can't do this	✅ Excellent — model trained on real audio
Precise parameter tweaking	✅ "Cutoff at 800Hz"	❌ "Make it darker" — probabilistic
Sound design exploration	🟡 Good — but limited by synth arch	✅ Endless — model has no boundaries
Low-end laptop (no GPU)	✅ Runs on anything	❌ Needs GPU
Real-time performance	✅ Instant parameter changes	🟡 5-30s generation latency